

Estructuras de Datos

Clase 10 – Tablas de Hash



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Tablas de Hash: Motivaciones

Se desea tener una implementación versátil de conjuntos, diccionarios y mapeos donde las operaciones de inserción, recuperación y eliminación tengan un tiempo esperado constante.

Hay dos implementaciones posibles:

1. Tabla de hash abierto (separate chaining en GT)
2. Tabla de hash cerrado (open addressing en GT)

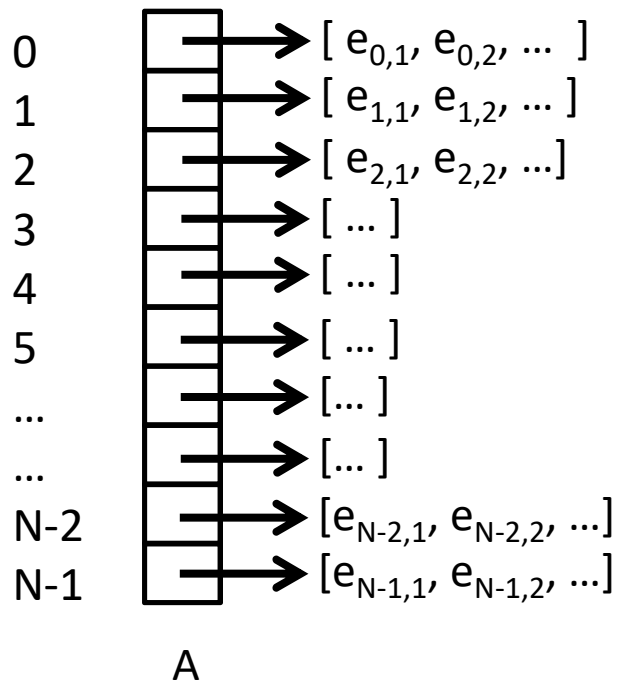
Discutiremos la implementación de mapeos.

La implementación de conjuntos y de diccionarios quedan a cargo del alumno.

Hash abierto

Hash Abierto: Definiciones

Arreglo de buckets: Un arreglo de cubetas para implementar una tabla de hash es un arreglo A de N componentes, donde cada celda de A es una colección de entradas (pares clave-valor) $e_{i,j}$.



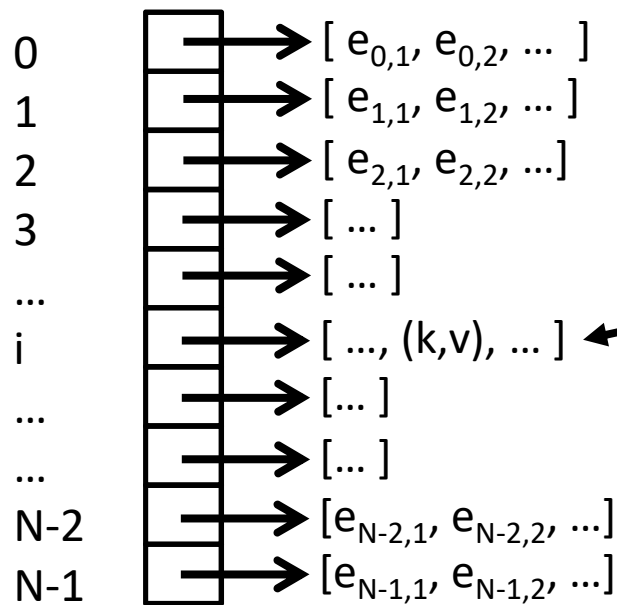
Cada colección $A[i]$ con $i=0,\dots,N-1$ se llama *bucket* o *cubeta*.

Nota simpática: *bucket* en inglés quiere decir *balde*.

Cuando los profesores éramos jóvenes, los únicos libros de computación editados en castellano eran de editoriales españolas, de ahí el término *cubeta*.

Hash Abierto: Definiciones

Función de hash h : Dada una clave k y un valor v , $h(k)$ es un entero en el intervalo $[0, N-1]$ tal que la entrada (k, v) se inserta en la cubeta $A[h(k)]$.



Dada la clave k , si $h(k) = i$,
entonces insertar (k, v) en $A[i]$.

(k, v)

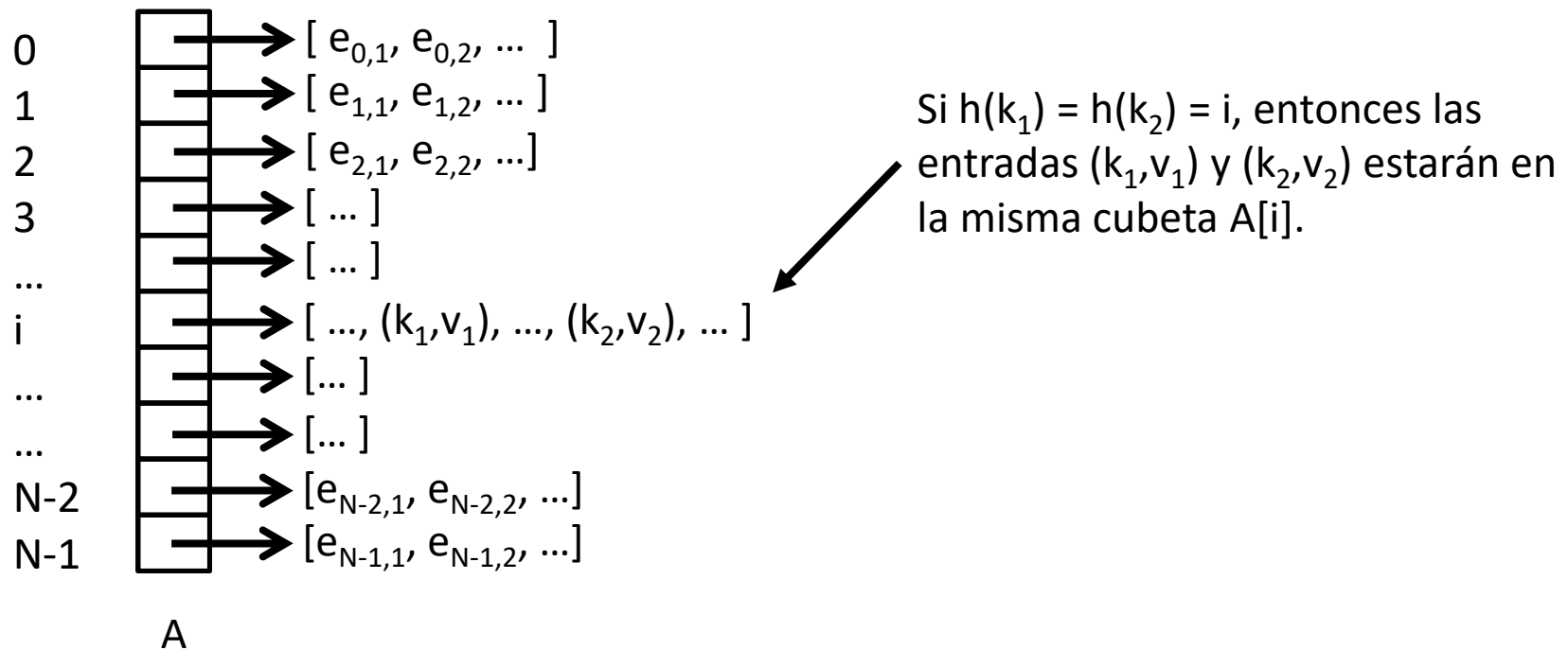
Dada la clave k , habrá que buscar
una entrada con clave k en la
cubeta i .

A

Hash Abierto: Definiciones

Colisión: Dadas dos claves k_1 y k_2 tales que $k_1 \neq k_2$, se produce una colisión cuando $h(k_1) = h(k_2)$.

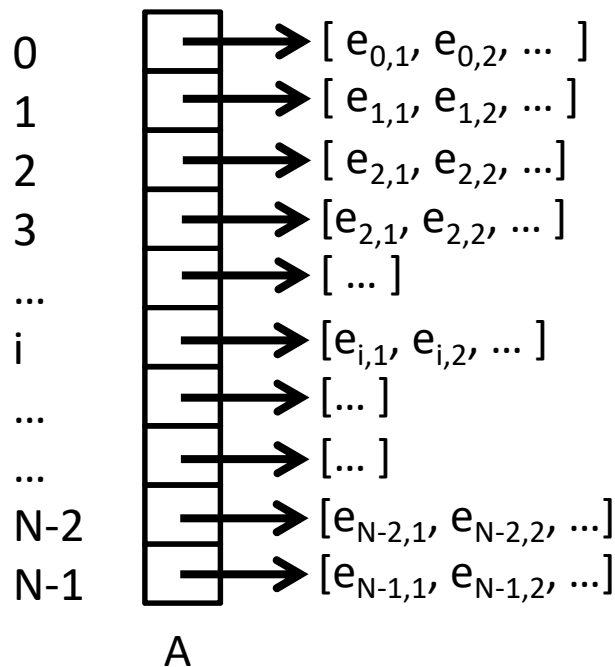
Nota: Las entradas de una cubeta tienen claves colisionadas.



Hash Abierto: Definiciones

“Buena” función de hash: Una función de hash h es “buena” si h distribuye las claves *uniformemente* en el intervalo $[0, N-1]$.

Intuitivamente, todas las cubetas tienen aproximadamente el mismo tamaño.



Si hay almacenadas n entradas en N cubetas, entonces el tamaño de cada cubeta es $O(n/N)$.

Nota: Para favorecer la distribución de claves uniforme, N debe ser *primo*.

Un número N es primo si solo es divisible por 1 y por N .

$N = 1, 2, 3, 5, 7, 11, 13, 17, \dots$

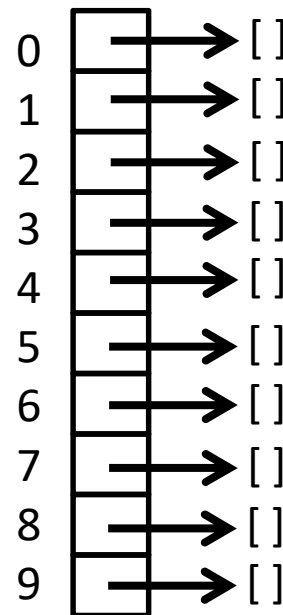
Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$ (N debiera ser primo, usamos 10 para hacer fácilmente los cálculos en el ejemplo).

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

`M ← new MapeoConHash();`

Nota: N en realidad debe ser primo, usaremos $N=10$ para facilitar los cálculos en el ejemplo.



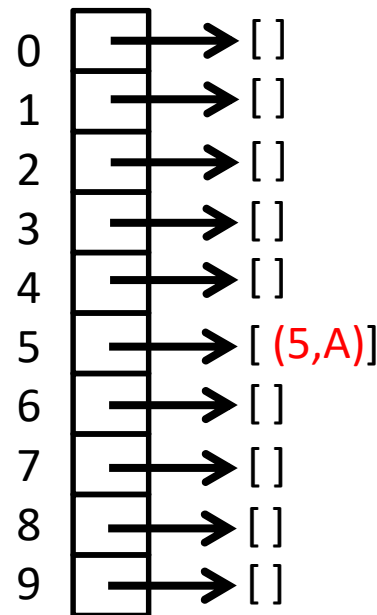
Hay 10 buckets (cubetas) numeradas de 0 a 9. Cada uno almacenará una colección de entradas. Cada bucket se inicializa con una lista vacía.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )
```



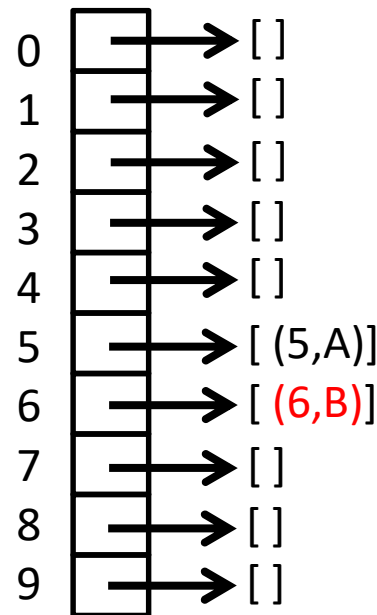
Como $h(5) = 5 \text{ mod } 10 = 5$, la entrada (5,A) se inserta al final de la lista $A[5]$.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )
```



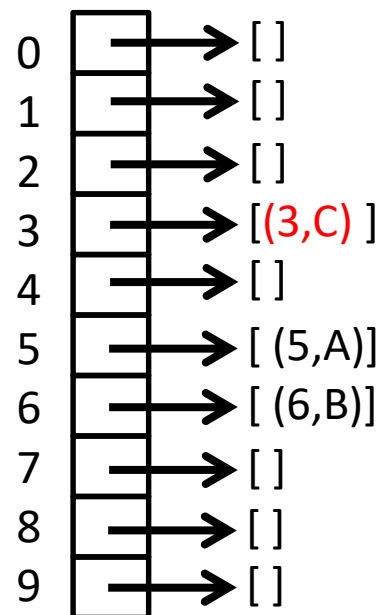
Como $h(6) = 6 \text{ mod } 10 = 6$, la entrada (6,B) se inserta al final de la lista $A[6]$.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )
```



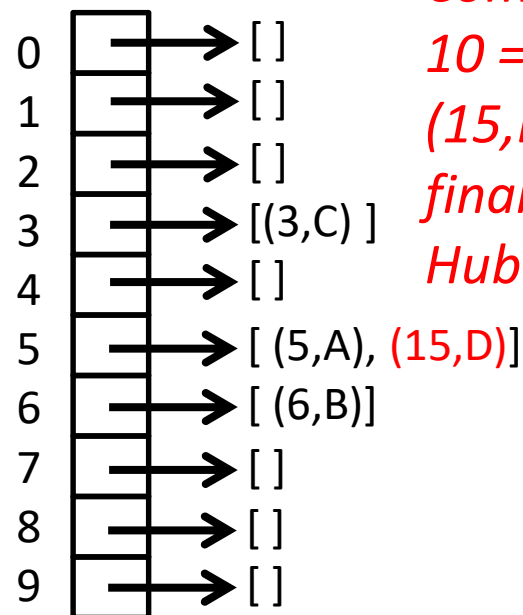
Como $h(3) = 3 \text{ mod } 10 = 3$, la entrada (3,C) se inserta al final de la lista $A[3]$.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )
```



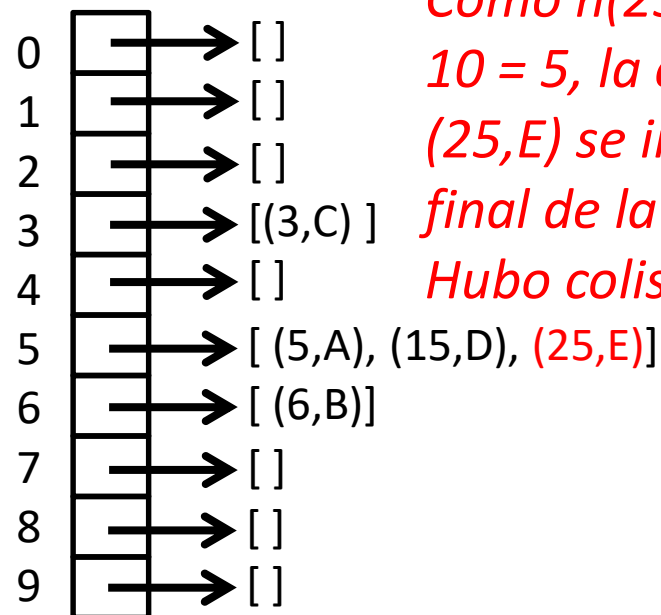
Como $h(15) = 15 \text{ mod } 10 = 5$, la entrada (15,D) se inserta al final de la lista A[5]. Hubo colisión.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )
```



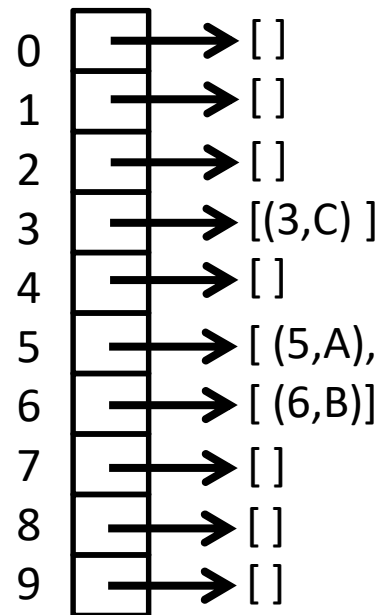
*Como $h(25) = 25 \text{ mod } 10 = 5$, la entrada (25,E) se inserta al final de la lista A[5].
Hubo colisión.*

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )
```



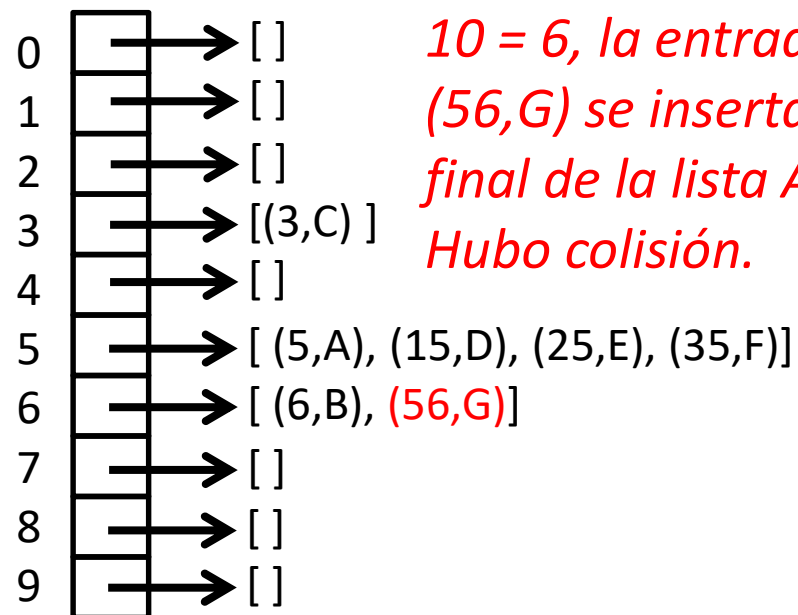
*Como $h(35) = 35 \text{ mod } 10 = 5$, la entrada (35,F) se inserta al final de la lista A[5].
Hubo colisión.*

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )  
M.put( 56, 'G' )
```



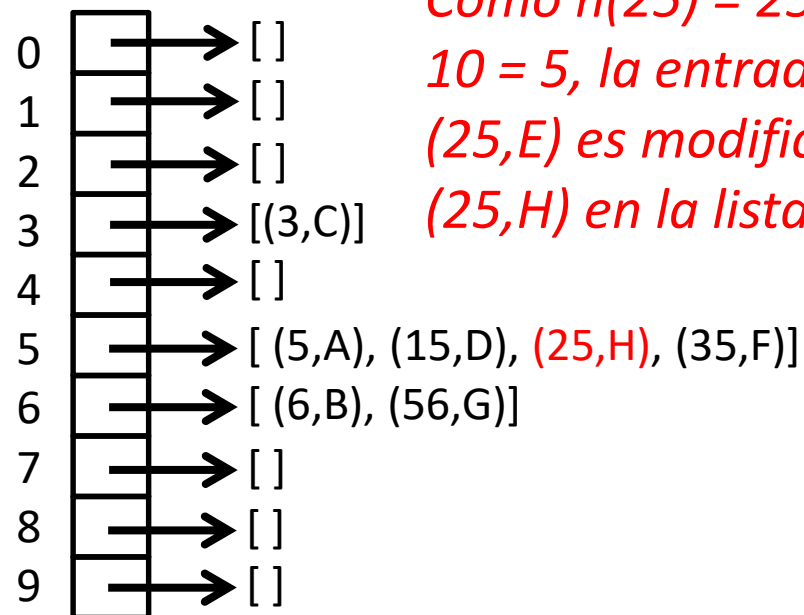
*Como $h(56) = 56 \text{ mod } 10 = 6$, la entrada (56,G) se inserta al final de la lista A[6].
Hubo colisión.*

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )  
M.put( 56, 'G' )  
M.put( 25, 'H' )
```



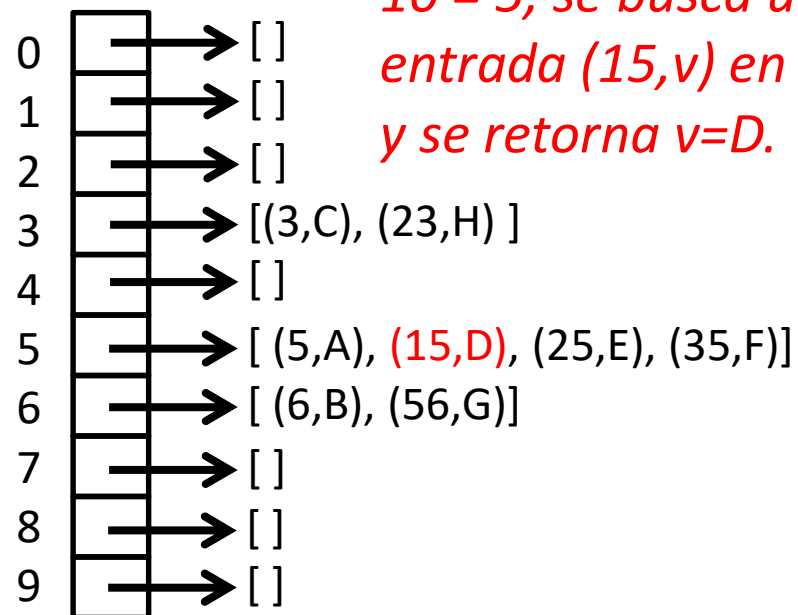
Como $h(25) = 25 \text{ mod } 10 = 5$, la entrada (25,E) es modificada a (25,H) en la lista A[5].

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )  
M.put( 56, 'G' )  
M.put( 23, 'H' )  
C ← M.get(15)
```



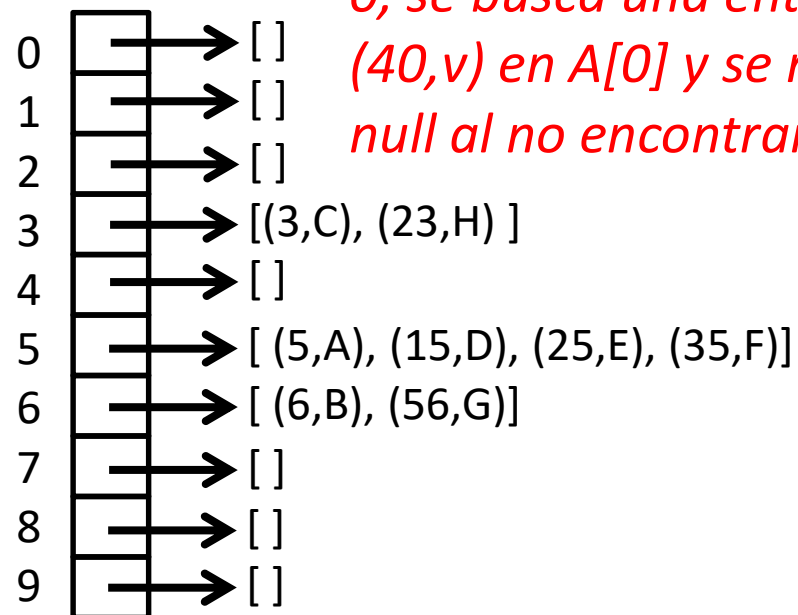
Como $h(15) = 15 \text{ mod } 10 = 5$, se busca una entrada (15,v) en $A[5]$ y se retorna $v=D$.

Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )  
M.put( 56, 'G' )  
M.put( 23, 'H' )  
C ← M.get(15);  C ← M.get(40)
```



Como $h(40) = 40 \text{ mod } 10 = 0$, se busca una entrada $(40,v)$ en $A[0]$ y se retorna null al no encontrarla.

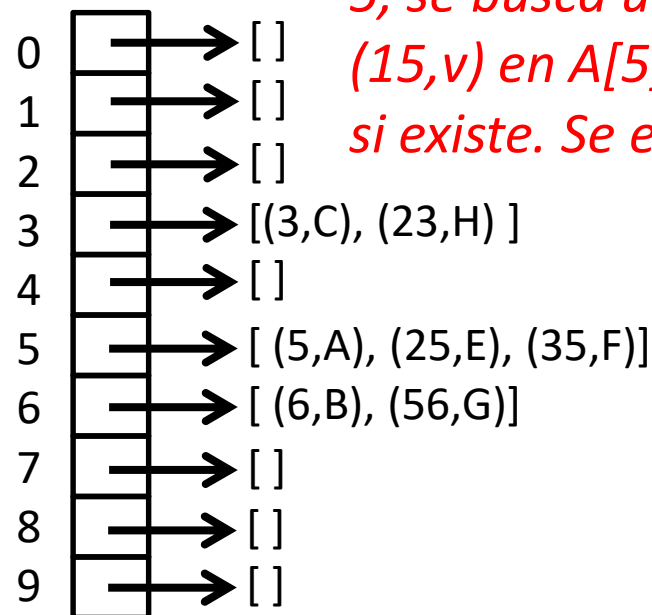
Ejemplo preliminar

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash();  
M.put( 5, 'A' )  
M.put( 6, 'B' )  
M.put( 3, 'C' )  
M.put( 15, 'D' )  
M.put( 25, 'E' )  
M.put( 35, 'F' )  
M.put( 56, 'G' )  
M.put( 23, 'H' )
```

```
C ← M.get(15); C ← M.get(40); M.remove(15)
```



Como $h(15) = 15 \text{ mod } 10 = 5$, se busca una entrada $(15,v)$ en $A[5]$ y se elimina si existe. Se elimina $(15,D)$

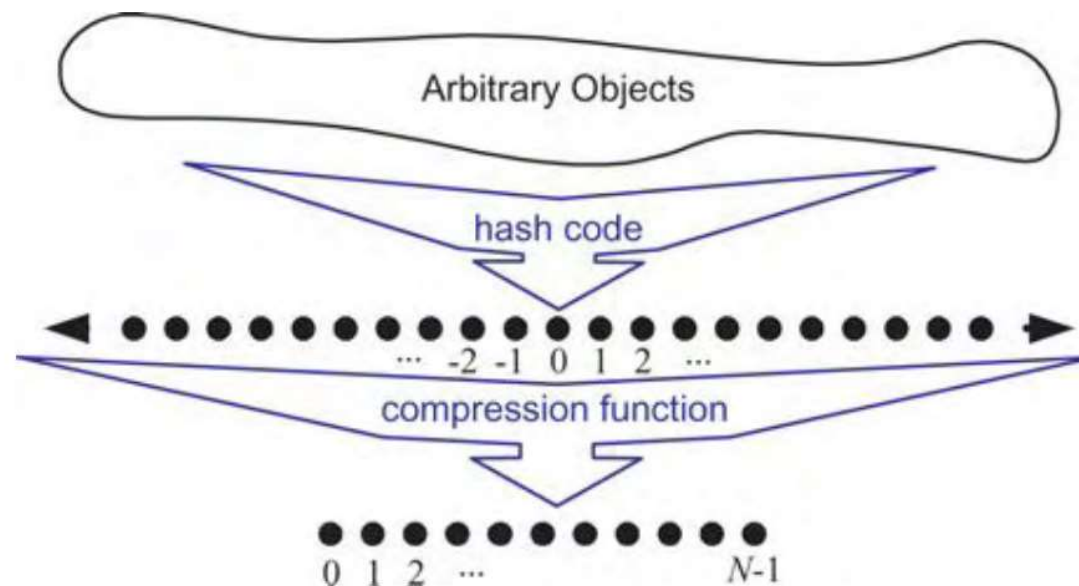
Problemas pendientes

- Hashing con claves genéricas:
 - ¿Cómo implementar $h(k)$ cuando k no es entero?
 - ¿Qué variantes de implementación hay para la función h ?
- Distribución uniforme de claves:
 - ¿Qué significa?
 - ¿Cómo lo implemento?

Hashing con claves genéricas

Paso 1 (hash code): Dada una clave k , obtener un número entero llamado *código de hash* a partir de k .

Paso 2 (función de compresión): A partir del código de hash obtener un valor entero entre 0 y $N-1$.



Paso 1: Códigos de hash

- La forma más sencilla es usar el método:

```
public int hashCode();
```

heredado de Object.

- Cuidado: En la implementación más simple *hashCode()* retorna la dirección de memoria del objeto con lo que dos objetos iguales (en el sentido de *equals*) tendrían distinto hashCode.
- Por ello la clase String redefine el método *hashCode()*.

Redefiniendo hashCode para una clase de usuario

```
public class Persona {  
    private String dni, numeroCelular, email;  
  
    // Redefino el método hashCode() heredado de Object para  
    // personalizarlo para la clase Persona usando los valores de atributos  
    // que permiten individualizar a una persona.  
    public int hashCode() {  
        return (dni + numeroCelular + email).hashCode();  
    }  
    ...  
}
```

Este mensaje hashCode() es a la clase String.



Códigos de hash para strings

- La forma más sencilla de implementar hashcode para strings consiste de sumar los códigos Ascii (o Unicode) de los caracteres del string.
- Ejemplo: El hash code de “ABBA” sería: $65+66+66+65 = 262$
- Sin embargo, el código de “BABA” sería el mismo, con lo cual habría una colisión.

Códigos de hash para strings

- Una aproximación mejor consiste en utilizar la posición del carácter considerado en un código polinomial.
- Si el string s es $(x_0, x_1, \dots, x_{k-1})$, dado un entero $a \neq 1$, se puede usar el hash code:

$$x_0a^{k-1} + x_1a^{k-2} + \dots + x_{k-2}a + x_{k-1}$$

Nota: El valor de “ a ” se debe encontrar experimentalmente hasta lograr una “buena” distribución de claves.

Códigos de hash para tipos numéricos

- Para el tipo Float:

Dado un float x existe un método de clase `Float.floatToIntBits(x)` que retorna un entero a partir de x.

Ejemplo:

- `Float.floatToIntBits(4.5697f): 1083325179`
- `Float.floatToIntBits(1.8e34f): 2019417596`
- Para los tipos byte, short, int, char: Castear a int.

Paso 2: Funciones de compresión

Dado $i = k.\text{hashCode}()$, se requiere en la implementación de:

- $\text{put}(k,v)$: obtener el número de cubeta a insertar la entrada (k,v)
- $\text{get}(k)$: obtener el número de cubeta para buscar una entrada con clave k .
- $\text{remove}(k)$: obtener el número de cubeta para buscar una entrada con clave k y eliminarla.

Implementaciones para calcular el número de cubeta a partir de i :

- Método de la división: $i \bmod N$ (N debe ser primo)
- Método MAD (loco) (por multiply, add & divide):

$$[(ai+b) \bmod p] \bmod N,$$

donde $a > 0$ y b son enteros random en $[0, p-1]$ y p es un primo, $p > N$.

Nota: a , b y p deben buscarse experimentalmente hasta lograr una distribución uniforme de claves.

Implementación de separate chaining

Clase Mapeo<K,V>

Atributos:

Map<K,V> [] A

int n { n es size, la cantidad de entradas del mapeo }

int N $\leftarrow$ 13 { o cualquier otro número primo }

Constructor Mapeo()

n $\leftarrow$ 0

A $\leftarrow$ (Map<K,V> []) new MapeoConLista[N]

for i $\leftarrow$ 0 **to** N-1 **do**

A[i] $\leftarrow$ new MapeoConLista<K,V>()

Implementación de separate chaining

Algoritmo get(k)

{ retorna el valor asociado con la clave k en el mapeo, o null si no hay una entrada con clave k en el mapeo }

return A[h(k)].get(k) { Delega en el get del mapeo implementado con lista. }

Algoritmo put(k, v)

{ Si hay una entrada en el mapeo con clave k, reemplaza el valor con v y retorna el viejo valor. Sino agrega la entrada (k,v) y retorna null }

t $\leftarrow$ A[h(k)].put(k,v) { Delega en el put del mapeo implementado con lista. }

if t = null **then** { k es una nueva clave }

 n $\leftarrow$ n+1 { incremento la cantidad de entradas en el mapeo }

return t

Algoritmo remove(k)

{ Retorna el valor asociado con la clave k del mapeo, o null si no hay una entrada con clave k }

t $\leftarrow$ A[h(k)].remove(k) { Delega en el mapeo implementado con lista. }

if t $\neq$ null **then** { k fue encontrado }

 n $\leftarrow$ n - 1 { decremento la cantidad de entradas del mapeo }

return t

¿Distribución uniforme de claves?

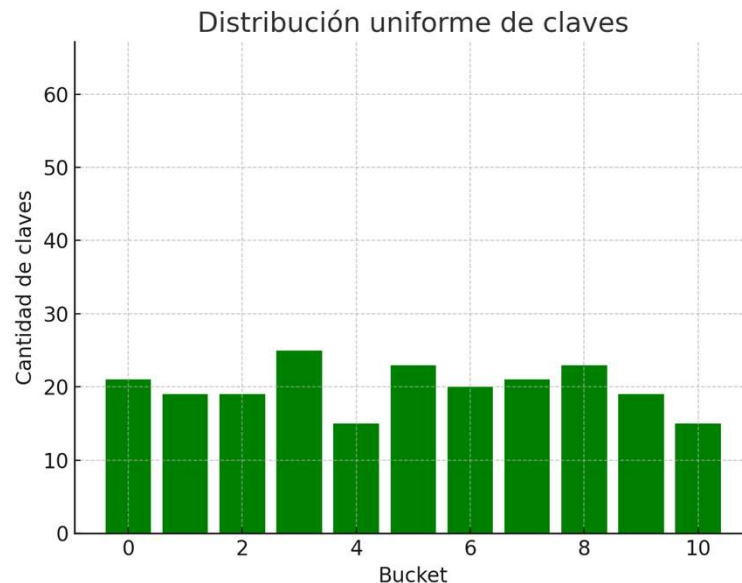
- Fenómeno aleatorio: Fenómeno que bajo mismas condiciones puede producir resultados diferentes.
- Evento: Resultado posible de un fenómeno aleatorio.
- Espacio muestral: Conjunto de eventos posibles.
- Ejemplo: Al arrojar un dado, los eventos posibles son que salga la cara 1, que salga la cara 2, ..., ó que salga la cara 6.
- Probabilidad: La probabilidad de un evento E, notada $p(E)$, es un número real entre 0 y 1.
- Ejemplo: Dado perfecto: $p(\text{salga la cara } i \text{ al arrojar el dado}) = 1/6$
- Frecuencia relativa: La probabilidad de un evento E se estima con su frecuencia relativa = cantidad de ocurrencias del evento E / cantidad de eventos ocurridos
- Ejemplo: Dado del casino: ¿Cuál es la probabilidad de obtener un 2 al tirar el dado? $p(\text{salga la cara 2 al arrojar el dado}) =$
cuántas veces salió el 2 / la cantidad de tiradas.

¿Distribución uniforme de claves?

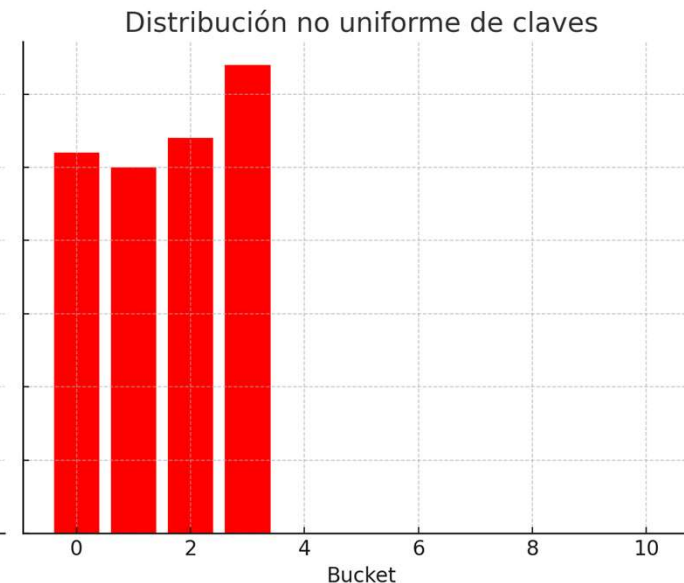
- Distribución uniforme: Todos los eventos tienen la misma probabilidad.
- Distribución uniforme de claves: La probabilidad de que una clave termine en un bucket determinado es constante y es igual a $1/N$, donde N =cantidad de buckets de la tabla de hash.
- ¿Cómo compruebo que tengo distribución uniforme de claves?:
 1. Tome un conjunto de n claves de muestra $k_1, k_2, \dots, k_n$
 2. Realice los n $put(k_1), put(k_2), \dots, put(k_n)$ en su tabla de hash
 3. Cuente cuántas claves (o entradas) hay en cada bucket
 4. Si hay distribución uniforme de claves, entonces en cada bucket debería haber aproximadamente n / N entradas.

Prueba empírica de la distribución de claves uniforme

- Si tengo una tabla de hash de 11 buckets, consigo un conjunto de valores de test y los inserto en la tabla.
- Escenarios posibles vistos como histograma:



Buena función de hash: Todos los buckets tienen aproximadamente la misma cantidad de entradas.



Mala función de hash: Los buckets 0 al 3 tienen todas las entradas y el resto quedaron vacíos.

Factor de carga λ

Sea n el número de entradas almacenadas en la tabla de hash.

Sea N el número cubetas de la tabla de hash (cantidad de componentes del arreglo de listas).

Supongamos $n = 200$ y $N = 11$, entonces $\lambda = n/N = 200/11 = 18,18$.

Esto quiere decir que cada bucket mide 19.

En el peor escenario, get, put y remove deben hacer 19 pasos, que es independiente de n y N .

Factor de carga en el libro $\lambda = 0,9$

- El factor de carga ($\lambda=n/N$) indica cuántos elementos hay en la tabla en relación con el número de buckets disponibles.
- En este caso, $\lambda=0.9$ sugiere que la tabla de hash está bastante llena.
- Si las claves están distribuidas de manera uniforme, entonces el número esperado de elementos en cada bucket es aproximadamente λ .
- Con $\lambda=0.9$, significa que en promedio, cada bucket tiene menos de un elemento (0.9 elementos por bucket en promedio).
- Esto implica que la mayoría de los buckets tendrán 0 o 1 elemento, ya que la distribución uniforme minimiza la posibilidad de que haya buckets con muchos elementos.
- No es una garantía absoluta (pueden existir colisiones debido a la naturaleza de la función hash), pero bajo una buena función hash y distribución uniforme, es altamente probable que las listas en los buckets estén vacías o contengan solo un elemento.

Rehash

- Al implementar put, hay que mantener el factor de carga $\lambda = n/N$ controlado.
- Si λ supera 0.9, hay que incrementar el tamaño de N para disminuir n/N .
- Esto requiere crear una tabla más grande (e.g. de tamaño $2*N$ y buscar el siguiente primo N' tal que $N' > 2*N$); luego insertar todas las entradas de la tabla vieja en la nueva tabla (*a resolver en la práctica*).
- Esta operación inevitablemente requiere recorrer toda la tabla vieja y asignar las entradas a su nueva ubicación en la tabla nueva.
- Ejemplo: Java usa $\lambda = 0.75$ como factor de carga en la clase HashMap.

Hash cerrado

Hash Cerrado (open addressing)

- En hash cerrado se tiene un arreglo de N buckets
- Cada bucket almacena a lo sumo una entrada.
- Dada una clave k y un valor v , la función de hash h indica cuál es la componente $h(k)$ del arreglo en la cual se almacena la entrada (k,v) .
- Dadas dos claves k_1 y k_2 con $k_1 \neq k_2$, si $h(k_1) = h(k_2)$ entonces se produce una colisión.
- Políticas para la resolución de colisiones:
 - Lineal (linear probing)
 - Cuadrática (quadratic probing)
 - Hash doble (double hashing)

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

$M \leftarrow \text{new MapeoConHash}();$

0	•
1	•
2	•
3	•
4	•
5	•
6	•
7	•
8	•
9	•

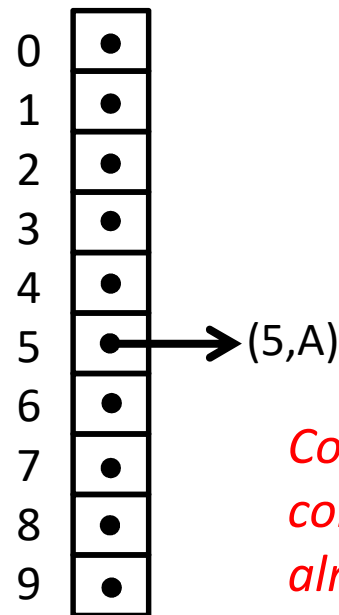
Inicialmente las componentes de la tabla de hash están inicializadas con null indicando que están vacías.

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )
```



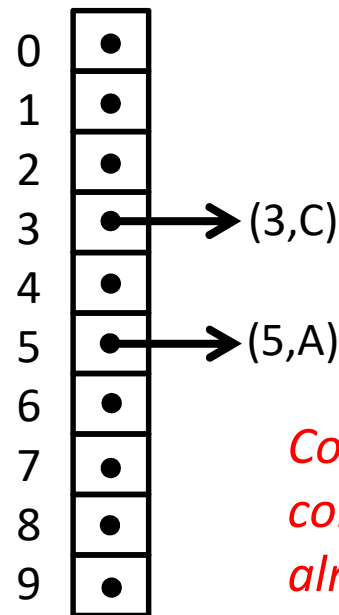
Como $h(5) = 5 \text{ mod } 10 = 5$ y la comp. 5 es null, directamente almaceno (5,A) en A[5].

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )
```



Como $h(3) = 3 \text{ mod } 10 = 3$ y la comp. 3 es null, directamente almaceno (3,C) en $A[3]$.

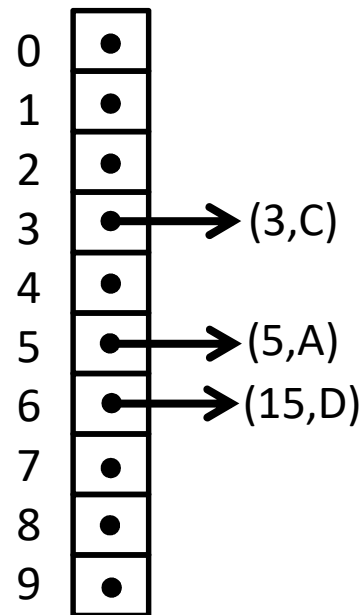
Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D')
```

Como $h(15) = 15 \bmod 10 = 5$ y la comp. 5 está ocupada, se produjo una colisión. Se busca una componente null o disponible en forma lineal y circular. Almaceno (15,D) en A[6].

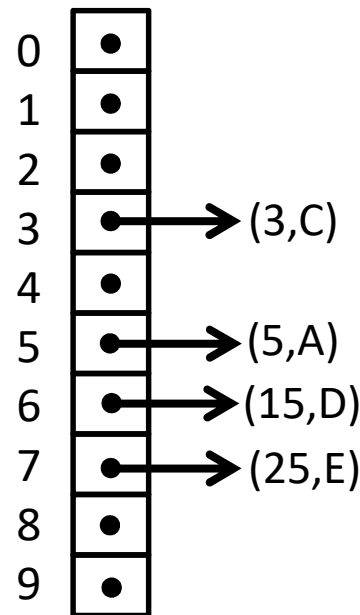


Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D' )  
M.put(25, 'E' )
```



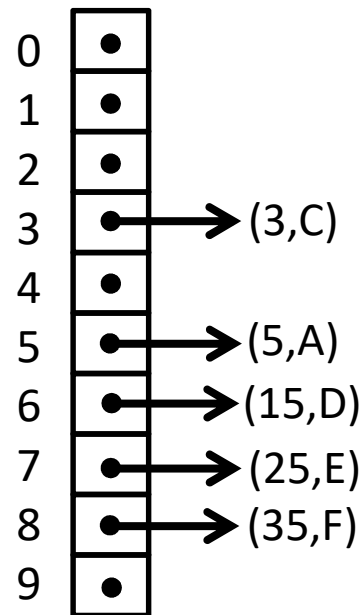
$h(25) = 25 \text{ mod } 10 = 5$, y la comp. 5 está ocupada, se produjo una colisión. Se busca una componente null o disponible en forma lineal y circular. Almaceno (25,E) en A[7].

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D' )  
M.put(25, 'E' )  
M.put(35, 'F' )
```



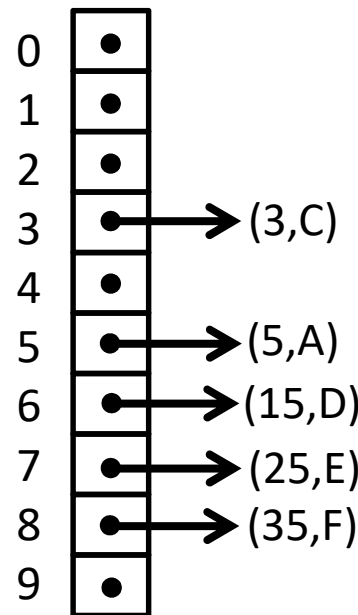
$h(35) = 35 \text{ mod } 10 = 5$, y la comp. 5 está ocupada, se produjo una colisión. Se busca una componente null o disponible en forma lineal y circular. Almaceno (35,F) en A[8].

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D' )  
M.put(25, 'E' )  
M.put(35, 'F' )  
v ← M.get(25)
```



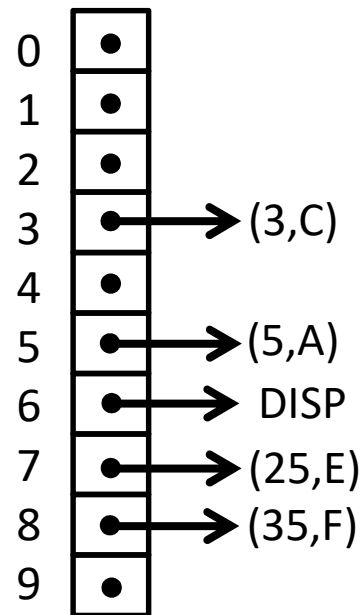
$h(25) = 25 \text{ mod } 10 = 5$, y la comp. 5 no tiene la clave 25, se busca linealmente la clave 25 hasta encontrarla o encontrar una componente null en forma lineal y circular. Encuentro la componente 7 y retorno el valor $A[7].getValue()$.

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()
M.put( 5, 'A' )
M.put( 3, 'C' )
M.put(15, 'D' )
M.put(25, 'E' )
M.put(35, 'F' )
v ← M.get(25)
M.remove(15)
```



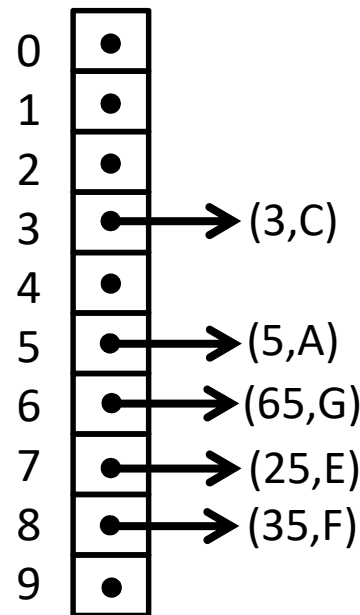
$h(15) = 15 \text{ mod } 10 = 5$, y la comp. 5 no tiene la clave 15, se busca linealmente la clave 15 hasta encontrarla o encontrar una componente null en forma lineal y circular. Encuentro la componente 6 y retorno el valor $A[6].getValue()$ pero marcando $A[6]$ como disponible. Si pusiera null no encontraría (25,E) ni (35,F).

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D' )  
M.put(25, 'E' )  
M.put(35, 'F' )  
v ← M.get(25)  
M.remove(15)  
M.put( 65, 'G' )
```



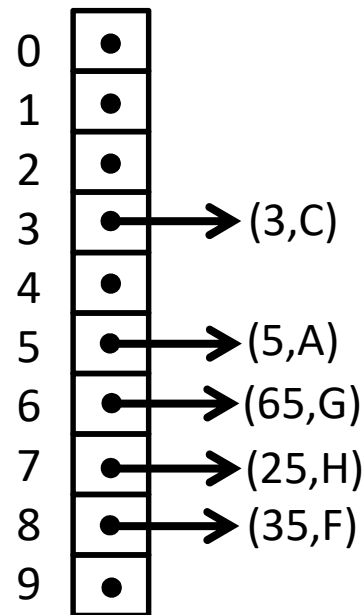
$h(65) = 65 \text{ mod } 10 = 5$, y la comp. 5 está ocupada (hubo colisión). Se busca linealmente hasta encontrar un null o un disponible en forma lineal y circular. Encuentro la componente 6 y allí almaceno (65,G).

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

```
M ← new MapeoConHash()  
M.put( 5, 'A' )  
M.put( 3, 'C' )  
M.put(15, 'D' )  
M.put(25, 'E' )  
M.put(35, 'F' )  
v ← M.get(25)  
M.remove(15)  
M.put( 65, 'G' )  
M.put(25,'H')
```



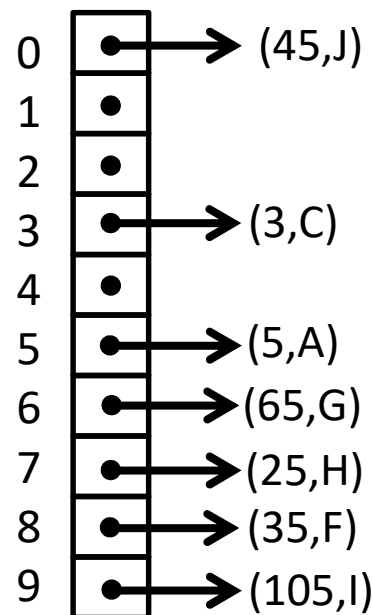
$h(25) = 25 \text{ mod } 10 = 5$, y la comp. 5 no tiene la clave 25, busco linealmente hasta un null o hasta un disponible o hasta hallar el 25, donde reemplazo el valor viejo E por el nuevo H y retorno el viejo E.

Ejemplo preliminar usando resolución lineal de colisiones

Supongamos que el tamaño del arreglo de bucket $N=10$ y que la función de hash es $h(k) = k \text{ MOD } N$.

Veamos qué sucede con la siguiente secuencia de operaciones sobre un mapeo de enteros en caracteres.

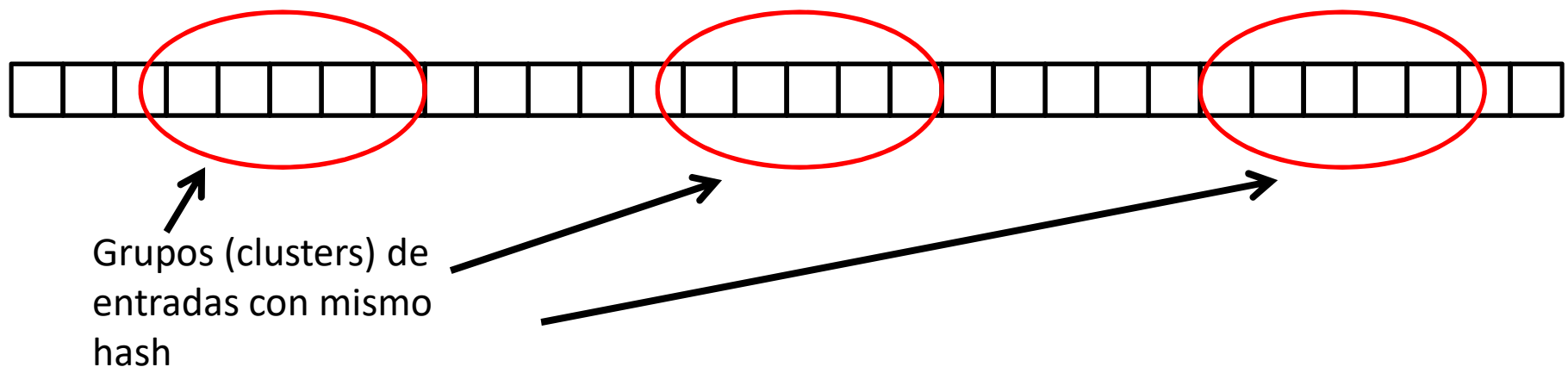
```
M ← new MapeoConHash()
M.put( 5, 'A' )
M.put( 3, 'C' )
M.put(15, 'D' )
M.put(25, 'E' )
M.put(35, 'F' )
v ← M.get(25)
M.remove(15)
M.put( 65, 'G' )
M.put(25,'H'); M.put( 105, 'I'); M.put(45, 'J')
```



La inserción de (105,I) y (45,J) pueblan el arreglo en forma circular.

Ventajas y desventajas

- El hashing lineal tiende a agrupar entradas con claves con mismo valor de hash.
- Si el arreglo es muy pequeño, se mezclan las claves k_i con distinto hash y la búsqueda se hace muy lenta porque degenera en buscar linealmente en un arreglo.
- Para hacer que esto funcione $N \gg n$ con lo que las claves se agrupan por su valor de hash
- GT sugiere que el factor de carga $\lambda = n/N < 0.5$



Hash cerrado con linear probing y $\lambda=0.5$

Si tengo distribución uniforme de claves y $\lambda=0.5$, quiere decir que $n/N < 0.5$.
Es decir, que el peor escenario esperado es algo similar a esto (donde x representa datos cargados):



Entonces, cualquier get y remove:

- o bien caen en una celda sin datos y salen con null
- o caen en una celda con datos y es el dato que busco y en un paso retornan el valor
- o caen en una celda con datos y no es el dato que busco y en 2 pasos se dan cuenta que el dato buscado no está (porque la celda adyacente está vacía).

Put tiene una situación similar:

- O cae en una celda vacía e inserta en un 1 paso
- O cae en una celda llena y encuentra el dato buscado y actualiza en 1 paso
- O cae en una celda llena y no encuentra el dato buscado y hace 1 paso más y va a la celda contigua que está vacía.

Otras políticas de resolución de colisiones: Resolución cuadrática (quadratic probing)

- Resolución lineal: Si $i=h(k)$, prueba iterativamente los buckets $A[(i+f(j)) \bmod N]$ para $j=0,1,2,\dots$ donde $f(j)=j$ hasta que encuentra un bucket vacío. Ej: si $h(k)=4$, prueba en los buckets 4,5,6,7,...
- Resolución cuadrática: Si $i=h(k)$, prueba iterativamente los buckets $A[(i+f(j)) \bmod N]$ para $j=0,1,2,\dots$ donde $f(j)=j^2$ hasta que encuentra un bucket vacío. Si $h(k)=4$, prueba en los buckets $4+0$, $4+1^2$, $4+2^2$, $4+3^2$, ...
- Ventaja: Evita los patrones de agrupamiento de hash lineal pero crea otro tipo que se llama “agrupamiento secundario”
- Problema: Si N no es primo, puede no encontrar un slot vacío aunque exista
- Problema: Si N es primo y el arreglo está lleno a la mitad o más (i.e. $n > N/2$), no va a encontrar un slot vacío por más que éste exista.

Otras políticas de resolución de colisiones: Hashing doble (double hashing)

- Definición: Si $i=h(k)$, y $A[i]$ está ocupado, se usa una segunda función de hash h' tal que prueba los buckets $A[(i + f(j)) \bmod N]$ para $j=1,2,3\dots$ donde $f(j) = j \cdot h'(k)$.
- La función $h'(k)$ no puede evaluar a 0.
- Se usa $h'(k) = q - (k \bmod q)$ para un primo $q < N$.
- N debe ser primo.

Comparación entre ambos enfoques

De acuerdo al libro:

- El hash cerrado ahorra algo de espacio con respecto al hash abierto.
- Los resultados experimentales muestran que el hash abierto es más rápido que los otros métodos.

Bibliografía

- Capítulo 9 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.